

Chapter 8 –Evaluation in IR

<https://chatgpt.com/share/69d9f2d5-1b1c-83e8-a839-4272a32c3648>

★ [8.1] Information retrieval system evaluation

An IR system is evaluated using a **test collection**, which includes three key parts:

1. **Document Collection** – a set of documents
2. **Queries (Information Needs)** – what users are searching for
3. **Relevance Judgments** – which documents are relevant

These three together allow us to **measure how well a system retrieves useful information**.

👉 Document Collection

- **dataset of documents** on which the IR system operates.
- should be **large and diverse**, because evaluation on a small dataset may give misleading results.
- **bigger collection** ensures more **realistic and reliable performance measurement**.

👉 Information Needs (Queries)

- What user wants to know
- a query is often just a **simplified version** of the real info need
 - **Query** → what user types

- **Information Need** → what user actually wants

👉 Relevance Judgments

Each document is marked as either:

- **Relevant**
- **Non-relevant**

for a given query.

This labeling helps in evaluating whether the system retrieves the **right documents** or not

👉 Gold Standard / Ground Truth

- **correct relevance judgments** are called the **gold standard (ground truth)**.
- usually created by **human experts**, and they serve as the **reference point** to evaluate system performance.

👉 Binary Relevance

two categories only:

- Relevant
- Not relevant

Binary classification is simpler and easier to evaluate

👉 Parameter Tuning Problem (Overfitting)

- IR systems often have **parameters (weights)** that can be adjusted
- If we tune these parameters directly on the **test collection**, the results will be:
 - **Biased**
 - **Over-optimistic**
- This is because the system becomes too tailored to that specific dataset.
- Wont perform well on unseen data

To avoid bias n keep it fair and generalized:

- Use a **development set** → for tuning parameters
- Use a **test set** → for final evaluation

[slides]

★ Different IR Models & Evaluation

> Which IR Model is the Best?

There are many Information Retrieval (IR) models (like Vector Space Model, Boolean Model, Probabilistic Model), and the question is: *which one is best?*

There is **no single universally best model** because performance depends on:

- **Type of data** (web pages, research papers, etc.)
- **Type of query** (short vs detailed)
- **User expectations**

A model that works well in one scenario may perform poorly in another. Therefore, IR systems are compared using **evaluation metrics (precision, recall, etc.)** on test collections rather than assuming one model is always superior.

› Best Ranking Function (dot product, cosine, etc.)

The ranking function determines **how documents are ordered in response to a query**.

- **Dot Product:** **Measures similarity by multiplying term weights** directly
- **Cosine Similarity:** **Normalizes document length** and **measures the angle between vectors**

Cosine similarity is often **preferred** because:

- It **removes bias** toward longer documents
- Provides a **fair comparison** between documents of different sizes

› Best Term Selection (Stopword Removal, Stemming...)

Term selection decides **which words to include or exclude** from indexing.

- **Stopword Removal:** Removes common words (e.g., *the, is, and*)
 - Helps **reduce noise and index size**
 - But **may remove useful context** in some queries
- **Stemming:** Reduces words to their **root form** (e.g., *running* → *run*)
 - Improves matching between related words
 - Not always produces a grammatically correct word
- **Lemmatization:** reduces words to their **dictionary (correct base) form**, called a *lemma*.
 - produces **meaningful, valid words**
 - More **accurate than stemming**
- **Case folding:** converts all text to a single case (usually lowercase).
 - Avoids treating same words as different
 - Simplifies matching
- **Tokenization:** process of splitting text into words (tokens).
- **Normalization:** standardizes text into a **consistent format**.
 - Removing punctuation + Handling accents (*café* → *cafe*)

There is no perfect choice — aggressive preprocessing may improve efficiency but can **hurt accuracy** in some cases.

> Best Term Weighting (TF, TF-IDF, etc.)

Term weighting determines **how important a word is** in a document.

- **TF (Term Frequency)**: Counts how often a term appears
 - **Higher frequency → more importance**
- **TF-IDF**: Combines term frequency with inverse document frequency
 - **Reduces weight of very common terms**
 - **Highlights rare but important words**

TF-IDF is widely used because it **balances**:

- **Local importance** (within document)
- **Global importance** (across collection)

👉 How Far Down the Ranked List Must a User Look?

This question focuses on **user effort and system effectiveness**.

In practice:

- Users usually only look at the **top few results** (e.g., first page)
- If relevant documents appear early → system is good
- If they appear far down → system is poor

This leads to evaluation measures like:

- **Precision at k (P@k)** → **relevance in top k results**

The goal of an IR system is to **rank relevant documents as high as possible**, minimizing how much the user has to search.

★ Difficulty in IR Evaluation

👉 Effectiveness depends on relevance

- **Effectiveness is related to relevancy of retrieved docs**
- If relevant documents appear at the top → system is effective
- If irrelevant documents dominate top results → system is ineffective

👉 Relevance is continuous (not strictly binary)

- In reality, relevance exists on a **spectrum**:
 - **Highly relevant**
 - **Partially relevant**
 - **Slightly relevant**
 - **Not relevant**
- Example: One document may fully answer a query, while another only gives partial information
- However, most IR systems simplify this into **binary (relevant / not relevant)** for easier evaluation
- This simplification can **lead to loss of detail and accuracy**

Even with binary decisions, it is often **hard to decide** if a document is relevant

- Reasons:
 - Document may only partially match the need
 - Information may be indirect or implicit

👉 Subjective nature of relevance

- Relevance depends on the **user's personal judgment**
- Two users with the same query may have **different expectations**
- Example:

- One user searching “Python” wants programming info
- Another wants information about the snake
- This subjectivity makes it **difficult to define** a **universal ground truth**

👉 **Situational nature of relevance**

- Relevance depends on the **user’s current situation or goal**
- The same document may be:
 - Relevant in one context
 - Irrelevant in another
- Example:
 - A beginner vs an expert searching the same topic will value different documents
- Therefore, relevance is **context-dependent**

👉 **Cognitive nature of relevance**

- Relevance is influenced by **human thinking, knowledge, and perception**
- Users interpret information differently based on:
 - Prior knowledge
 - Understanding level
 - Expectations
- A document that is **clear to one** user may be **confusing or irrelevant to another**
- This makes evaluation dependent on **human cognition**

👉 **Dynamic nature of relevance**

- Relevance **changes over time**
- Reasons:

- User needs evolve
- New information becomes available
- Old information becomes outdated
- Example: A news article may be highly relevant today but not after a few months
- This makes it **hard to maintain a fixed evaluation standard**

★ Info need translation to a query:

- Information Need
 - Drinking red wine is more effective at reducing your risk of heart attacks than white wine.
- Query
 - wine and red and white and heart and attack and effective

> Info need:

- Information need refers to the **actual problem or requirement in the user's mind**
- It is usually **detailed, specific, and expressed in natural language**

> query:

- query is the **simplified version of the information need** that the user types into the system
 - It usually contains **keywords instead of full sentences**
-
- IR systems must try to **infer the real information need from the query**
 - If the query is vague or incomplete, the system may return **irrelevant results**
 - Effective systems aim to **bridge this gap** using techniques like query expansion and ranking

★ Ad hoc Information Retrieval

👉 Ad hoc Information Retrieval (Definition)

- Ad hoc IR → **retrieving documents for new, unseen queries without prior knowledge**
- The system does not know the query in advance and must **process it on the spot**
- It focuses on **general-purpose search**, like web search engines
- Each query is treated independently, with no learning from past queries

👉 Need for Standard Evaluation

- To measure how effective an ad hoc IR system is, we need a **standard evaluation framework**
- Without a standard method, it is difficult to **compare different systems fairly**
- This ensures results are **consistent, reliable, and reproducible**
- For this we need the following:
 - 1) **Doc collection**
 - 2) **Test queries**
 - 3) **Relevance judgements**

★ Evaluation in IR

- Evaluation measures are used to determine **how well search results satisfy the user's intent**
- helps in improving system design and performance
- Evaluation allows us to **compare different IR systems or models**
- By using the same dataset and queries, we can see **which system performs better**
- Metrics like **precision** and **recall** are commonly used for this purpose

👉 Evaluation as a Research Area

- Evaluation is not fixed; it is an **active area of research in IR**
- Researchers **continuously develop**:
 - **New evaluation metrics**
 - **Better relevance models**
 - **Improved testing methodologies**
- This is **because user behavior and data are constantly evolving**

👉 Evolution of Evaluation Datasets

- Initially, evaluation used:
 - Small datasets (~100 documents, ~30 queries)
- Over time, datasets have grown significantly to:
 - Very large collections (approaching **web-scale**)
- Larger datasets provide:
 - More realistic testing
 - Better generalization of results

★ Standard IR collections

- The Cranfield collection.
 - Collected in the United Kingdom starting in the late 1950s, it contains 1398 abstracts of aerodynamics journal articles, a set of 225 queries, and exhaustive relevance judgments of all (query, document) pairs.
- 20 Newsgroups
 - It consists of 1000 articles from each of 20 Usenet newsgroups (the newsgroup name being regarded as the category).

- Cross Language Evaluation Forum (CLEF)
 - This evaluation series has concentrated on EU languages and cross-language information retrieval.
- Reuters-21578 and Reuters-RCV1
 - For text classification, the most used test collection has been the Reuters-21578 collection of 21578 newswire articles
- WebKB
 - This data set contains WWW-pages collected from computer science departments of various universities in January 1997
- Modern IR Collections
 - TREC
 - SemEval
 - LSHTC
 - CLEF
 - MediaEval

Collection	Main use	What to remember
Cranfield	Early classic IR collection	1398 abstracts, 225 queries, exhaustive relevance judgments; historically

Chapter 8 Exam Notes | Evaluation in Information Retrieval

Collection	Main use	What to remember
		important but now too small for serious modern evaluation.
TREC	Most famous large IR evaluation series	NIST test bed since 1992; early ad hoc collections total about 1.89 million documents and 450 information needs.
GOV2	Large web collection for research	About 25 million web pages; much larger than earlier research collections.
NTCIR	East Asian and cross-language IR	Useful when queries and documents are in different languages.
CLEF	European cross-language IR	Focuses on European languages and multilingual retrieval.
Reuters-21578 / RCV1	Text classification collections	Reuters-21578 is older; RCV1 is much larger and richly annotated.
20 Newsgroups	Text classification benchmark	20 categories from Usenet; widely used in classification experiments.

★ Image Captioning

- **automatically generating a textual description for an image**

TextCaps dataset

v0.1

Training set

- [109,765 captions \(173MB\)](#)
- [21,953 images \(6.6GB\)](#)
- [Rosetta OCR tokens \[v0.2\]](#)

Validation set

- [15,830 captions \(25MB\)](#)
- [3,166 images](#)
- [Rosetta OCR tokens \[v0.2\]](#)

Test set

- [Metadata \(6.5MB\)](#)
- [3,289 images \(926MB\)](#)
- [Rosetta OCR tokens \[v0.2\]](#)

- **TextCaps dataset (v0.1)**, which is a specialized dataset used for **Image Captioning**
- **Training Set:** Used to "teach" the model. With **109,765 captions** and **21,953 images**, this is where the model learns the relationship between visual features and descriptive text.
- **Validation Set:** Used for **Hyperparameter Tuning**. It helps the IR researcher decide which version of a model is performing best without "spoiling" the final test results.
- **Test Set:** The "final exam." It contains **3,289 images** and associated metadata to evaluate how well the retrieval or captioning system performs in the real world.

★ Evaluation of Unranked Retrieval Sets

- 1) **Precision**
- 2) **Recall**
- 3) **F-measure / F-score**
- 4) **Fall-out**

👉 Unranked (Flat) Result Set

- In some IR systems, results are returned as a **simple set of documents (no ranking/order)**
 - The system **only decides whether a document** is **relevant enough to include**, not where to place it
 - **All retrieved documents** are treated **equally important**
 - Example: A system returns {D1, D2, D3} without ترتيب (no ranking like top, second, etc.)
-

👉 Result Set as a “Set” (No Duplicates)

- The retrieved results are considered a **set**, meaning:
 - **No duplicate documents**
 - Each document appears only once
-

👉 Relevant vs Retrieved Documents

- For any query, there are two important sets:
 - **Relevant documents (R)** → all **correct documents** in the collection
 - **Retrieved documents (A)** → **documents returned** by the system
 - The system may retrieve:
 - Some relevant documents
 - Some non-relevant documents
 - The overlap between these sets is crucial for evaluation
-

👉 Possible Scenarios (Very Important Concept)

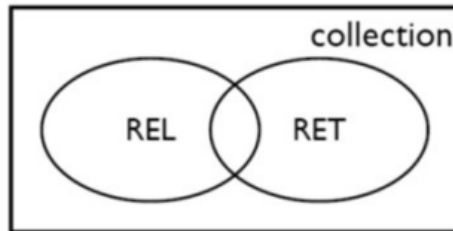
- Retrieved & Relevant →  Correct results
- Retrieved & Not Relevant →  (noise)
- Not Retrieved & Relevant →  missed important documents
- Not Retrieved & Not Relevant →  correctly ignored

👉 These four cases form the basis of **precision and recall**

👉 Precision (P)

- Precision measures **how accurate the retrieved results are**
- Focuses on **quality of results**
- Formula:

- **Precision = (Relevant Retrieved) / (Total Retrieved)**
- It answers: *Out of all retrieved documents, how many are actually relevant?*



$$\mathcal{P} = \frac{|RET \cap REL|}{|RET|}$$

Example:

- Retrieved documents = 10
- Relevant among them = 7

👉 Precision = $7 / 10 = 0.7$ (70%)

- **High precision** means:
 - **Few irrelevant documents**
 - System is **accurate**
- A system with high precision:
 - Rejects non-relevant documents effectively
- But it **may still miss some relevant documents (low recall)**

👉 Recall (R)

- Recall measures **how many relevant documents are found**
- Focuses on **completeness of results**
- Formula:
 - **Recall = (Relevant Retrieved) / (Total Relevant in collection)**

- It answers: *Out of all relevant documents, how many did the system retrieve?*

Example:

- Total relevant documents = 20
- Retrieved relevant = 7

👉 Recall = $7 / 20 = 0.35$ (35%)

- **High recall** means:
 - **Most relevant documents are found**
 - System has good **coverage**
- includes many irrelevant docs? (**low precision**)

$$\mathcal{R} = \frac{|RET \cap REL|}{|REL|}$$

👉 Precision vs Recall Trade-off

- **Increasing precision** often **decreases recall**
- **Increasing recall** often **decreases precision**

Example:

- If system retrieves only top 3 highly relevant docs → high precision, low recall
- If system retrieves 100 docs (including many irrelevant) → high recall, low precision [high recall cause zyada se zyada relevant retrieve/cover hoye]

👉 Combined Example

- Total documents = 100
- Relevant documents = 20

- Retrieved documents = 10
- Relevant retrieved = 6

👉 Precision = $6 / 10 = 0.6$

👉 Recall = $6 / 20 = 0.3$

- Interpretation:
 - System is **fairly accurate (precision)**
 - But misses many relevant documents (low recall)

		<u>relevant</u>		
		Rel	NotRel	
<u>retrieved</u>	Ret	Ret_{Rel}	Ret_{NotRel}	$Ret = Ret_{Rel} + Ret_{NotRel}$
	NotRet	$NotRet_{Rel}$	$NotRet_{NotRel}$	$NotRet = NotRet_{Rel} + NotRet_{NotRel}$

$$\text{Relevant} = Ret_{Rel} + NotRet_{Rel}$$

$$\text{Not Relevant} = Ret_{NotRel} + NotRet_{NotRel}$$

$$\text{Total \# of documents available } N = Ret_{Rel} + NotRet_{Rel} + Ret_{NotRel} + NotRet_{NotRel}$$

- Precision: $P = Ret_{Rel} / \text{Retrieved}$
- Recall: $R = Ret_{Rel} / \text{Relevant}$

$$P = [0, 1]$$

$$R = [0, 1]$$

[relevant and retrieved are reversed here | position badal gayi]

	Retrieved	Not retrieved	
Relevant	w=3	x=2	Relevant = w+x= 5
Not relevant	y=3	z=2	Not Relevant = y+z = 5
	Retrieved = w+y = 6	Not Retrieved = x+z = 4	
	Total documents N = w+x+y+z = 10		

- Precision: $P = w / w+y = 3/6 = .5$
- Recall: $R = w / w+x = 3/5 = .6$

› Precision vs Recall

- inversely proportional

👉 Types of Errors in IR Systems

- IR systems can make two main types of errors:
 - **False Positive** → system retrieves a **non-relevant document**
 - **False Negative** → system **fails to retrieve a relevant document**
- These errors directly affect how well the system performs in terms of **precision and recall**

👉 False Positive Error (Impact on Precision)

- This increases the number of **irrelevant documents in the retrieved set**
- As a result:

- **Precision decreases** (because precision measures correctness of retrieved results)

Example:

- Retrieved = 10 documents
- Relevant = 6, Non-relevant = 4

👉 Precision = $6 / 10 = 0.6$

- Those 4 irrelevant docs are **false positives**, lowering precision
-

👉 **False Negative Error (Impact on Recall)**

- This **reduces the number of relevant documents retrieved**
- As a result:
 - **Recall decreases** (because recall measures coverage of relevant documents)

Example:

- Total relevant = 20
- Retrieved relevant = 6 (missed 14)

👉 Recall = $6 / 20 = 0.3$

- The 14 missed documents are **false negatives**, **lowering recall**
-

👉 **Relation Summary :**

Reducing one type of error often increases the other, creating a **trade-off**

👉 Precision-Critical Tasks

- **accuracy is more important than completeness**
 - Key characteristics:
 - **Time matters a lot** → users want quick, correct answers
 - Some relevant documents can be missed (**tolerance exists**)
 - Information is often **redundant** (many sources give same info)
 - Example:
 - **Web search**
 - Demand:
 - **Very high performance**, fast and accurate
 - Optimization:
 - Focus on **ranking top results correctly**
-

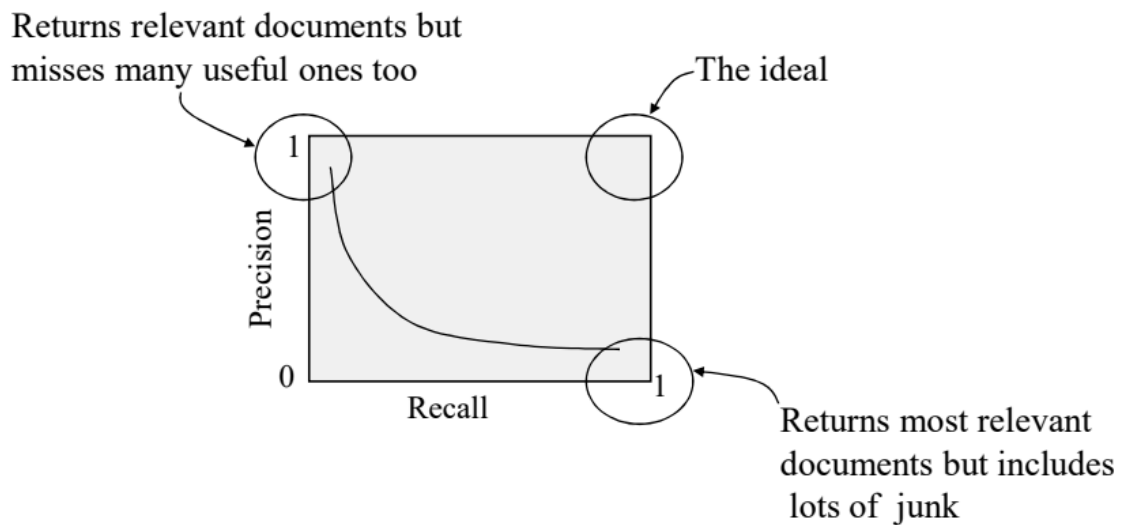
👉 Recall-Critical Tasks

- **finding all relevant information is critical**
 - Key characteristics:
 - **Time is less important**
 - **No tolerance** for missing documents
 - Information is **rare or non-redundant**
 - Example:
 - **Legal search / Patent search**
 - Demand:
 - **Moderate speed**, but very high completeness
 - Optimization:
 - Focus on **retrieving as many relevant documents as possible**
-

👉 Precision vs Recall Trade-off

- Improving precision:
 - Reduce false positives
 - But may miss relevant documents → lower recall
- Improving recall:
 - Retrieve more documents
 - But may include irrelevant ones → lower precision

👉 Systems must balance both depending on the **application needs**



Precision and Recall are inverse proportional

High Precision, Low Recall (Top Left)

- **The Behavior:** The system is very "picky." It only returns results it is extremely confident about.
- **The Result:** You get zero "junk" (high precision), but you miss out on many other relevant documents that were slightly less obvious to the algorithm.

Low Precision, High Recall (Bottom Right)

- **The Behavior:** The system is "greedy." it tries to grab every possible relevant document it can find.
- **The Result:** You find almost everything useful that exists (high recall), but you also pull in a massive amount of irrelevant "noise" or junk.

The Ideal (Top Right)

- **The Behavior:** This is the "Holy Grail" of IR.
- **The Result:** The system finds **every** relevant document (Recall = 1) and **only** relevant documents (Precision = 1).

[^ not possible]

F-Measure

👉 Why F-Measure is Needed

- Precision and Recall often **conflict with each other**
 - Increasing precision → may decrease recall
 - Increasing recall → may decrease precision
 - So we need a **single measure that combines both**
 - F-measure provides a **balanced evaluation of system performance**
 - It ensures both **accuracy and completeness are considered**
-

👉 F-Measure Formula

$$F = \frac{(\beta^2 + 1) \cdot P \cdot R}{(\beta^2 \cdot P + R)}$$

- Where:
 - **P = Precision**
 - **R = Recall**
 - **β (beta) controls importance of precision vs recall**
 - **$\beta < 1$ → Precision favoured**
 - **$\beta > 1$ → Recall favoured**
 - **$\beta = 1$ → equally important**
-

👉 F1-Score (Most Common Case)

- When **$\beta = 1$** , precision and recall are **equally important**

$$F1 = \frac{2PR}{P + R}$$

- This is the **harmonic mean** of precision and recall
 - It gives a **balanced score only when both P and R are high**
-

👉 Example (Step-by-Step)

- Suppose:
 - Precision (P) = 0.6
 - Recall (R) = 0.3

👉 Using F1 formula:

$$F1 = \frac{2 \cdot 0.6 \cdot 0.3}{0.6 + 0.3} = \frac{0.36}{0.9} = 0.4$$

👉 Interpretation:

- Even though precision is decent (0.6), low recall (0.3) **pulls the score down**
 - F-measure **penalizes imbalance between P and R**
-

👉 Role of β (Beta)

- β controls **which metric is more important**

Case 1: $\beta = 1$

- Precision = Recall importance
- Balanced evaluation
- Most commonly used → **F1-score**

Case 2: $\beta < 1$ (e.g., $\beta = 0.5$)

- **Precision** is favored more
- Useful when:
 - False positives are costly
 - Example: Web search (user wants accurate top results)

Case 3: $\beta > 1$ (e.g., $\beta = 2$)

- **Recall is favored more**
 - Useful when:
 - Missing relevant documents is dangerous
 - Example: Medical or legal search
-

👉 Example with β (Conceptual)

- Suppose:
 - $P = 0.8, R = 0.4$
- If $\beta = 0.5$ (precision-focused):
 - F-score will be **closer to precision (0.8)**
- If $\beta = 2$ (recall-focused):
 - F-score will be **closer to recall (0.4)**

👉 So β shifts the importance toward one metric

★ Problems with Precision and Recall

- Precision and recall **do not consider** total number of **irrelevant documents**
- Recall is **undefined** when there is **no relevant document in the collection**.

$$\text{Recall} = \frac{\text{Relevant Retrieved}}{\text{Total Relevant}}$$

- If **total relevant = 0**, then:
 - Division by zero → **undefined**
- This situation occurs when:
 - No document in the collection is relevant to the query
- Precision is **undefined** when **no document is retrieved**.

$$\text{Precision} = \frac{\text{Relevant Retrieved}}{\text{Total Retrieved}}$$

- If system retrieves **no documents**, then:
 - Division by zero → **undefined**
- Happens when system is too strict or fails to return results

★ Fallout Rate

Fallout measures the proportion of **non-relevant documents** that are **incorrectly retrieved**

- focuses specifically on **errors among irrelevant documents**
- While precision looks at retrieved results, fallout looks at the **entire set of irrelevant documents in the collection**

$$\text{Fallout} = \frac{\text{Non-relevant Retrieved}}{\text{Total Non-relevant Documents in Collection}}$$

- Numerator → irrelevant documents retrieved (**false positives**)
- Denominator → all irrelevant documents موجود in the collection

👉 Example (Step-by-Step)

- Total documents = 100
- Relevant documents = 20
- Non-relevant documents = 80
- System retrieves = 10 documents
 - Relevant retrieved = 6
 - Non-relevant retrieved = 4

$$\text{Fallout} = \frac{4}{80} = 0.05$$

- Fallout = **0.05 (5%)**

👉 Interpretation:

- Only 5% of all irrelevant documents were mistakenly retrieved
 - **Lower fallout = better system (fewer false positives)**
-

👉 Relation with Precision

- Precision also considers non-relevant retrieved but:
 - Precision = based only on **retrieved set**
 - Fallout = based on **entire collection**
- So fallout gives a **broader view of system errors**

★ Evaluation of Ranked Retrieval Results

- 1) Precision –Recall Curve
- 2) Average Precision
- 3) Mean Average Precision (MAP)
- 4) Cumulative Gain
- 5) Discount Cumulative Gain
- 6) Normalized Discount Cumulative Gain

👉 Set-based Measures (Limitation)

- Precision, Recall, and F-measure are **set-based measures**
 - They treat retrieved documents as an **unordered set (no ranking considered)**
 - This means:
 - A **relevant document at position 1 and position 100 are treated the same**
 - In reality, **ranking matters** because **users care more about top results**
-

👉 Need for Ranked Evaluation

- In **ranked retrieval** (like search engines), **documents** are **returned in a sorted order of relevance**
 - Users usually:
 - Look only at the **top few results**
 - Therefore, evaluation must consider: **Position of relevant documents**, not just whether they are retrieved
-

👉 Top-k Retrieved Documents

- Instead of evaluating the whole set, we **consider only** the **top k results**
 - Example:
 - Top 5, Top 10, Top 20 documents
 - This reflects **real user behavior**, as users rarely go beyond first few results
-

👉 Precision at k (P@k)

- Precision can be calculated at a specific rank k:

$$P@k = \frac{\text{Relevant documents in top } k}{k}$$

Example:

- Top 5 results → 3 are relevant

$$P@5 = \frac{3}{5} = 0.6$$

- This tells us **how accurate the top results** are

👉 System Can Return Any Number of Results

- Unlike unranked systems, ranked systems can:
 - **Return large lists of documents**
 - But evaluation focuses on:
 - **Top portion of results**
 - Because lower-ranked documents are usually **ignored by users**
-

1) Precision-Recall Curve

- A precision-recall curve shows the **relationship between precision and recall at different thresholds**
 - As we **move down the ranked list**:
 - **Recall increases** (more relevant docs found)
 - **Precision usually decreases** (more non-relevant docs included)
-

👉 How It is Computed

- We go through the ranked list step-by-step:
 - **At each position, compute precision and recall**
 - Then plot:
 - **Recall on x-axis**
 - **Precision on y-axis**
-

👉 Importance of Precision-Recall Curve

- Helps **visualize trade-off between precision and recall**
- better system has a curve **Closer to top-right corner** (high precision & high recall)

[Slide example]

n	doc #	relevant
1	588	x
2	589	x
3	576	
4	590	x
5	986	
6	592	x
7	984	
8	988	
9	578	
10	985	
11	103	
12	591	
13	772	x
14	990	

Let total # of relevant docs = 6

Check each new recall point:

$R=1/6=0.167$; $P=1/1=1$

$R=2/6=0.333$; $P=2/2=1$

$R=3/6=0.5$; $P=3/4=0.75$

$R=4/6=0.667$; $P=4/6=0.667$

$R=5/6=0.833$; $P=5/13=0.38$

Missing one
relevant document.
Never reach
100% recall

1. **Recall (R):** $\frac{\text{Relevant docs found so far}}{\text{Total relevant in collection (6)}}$

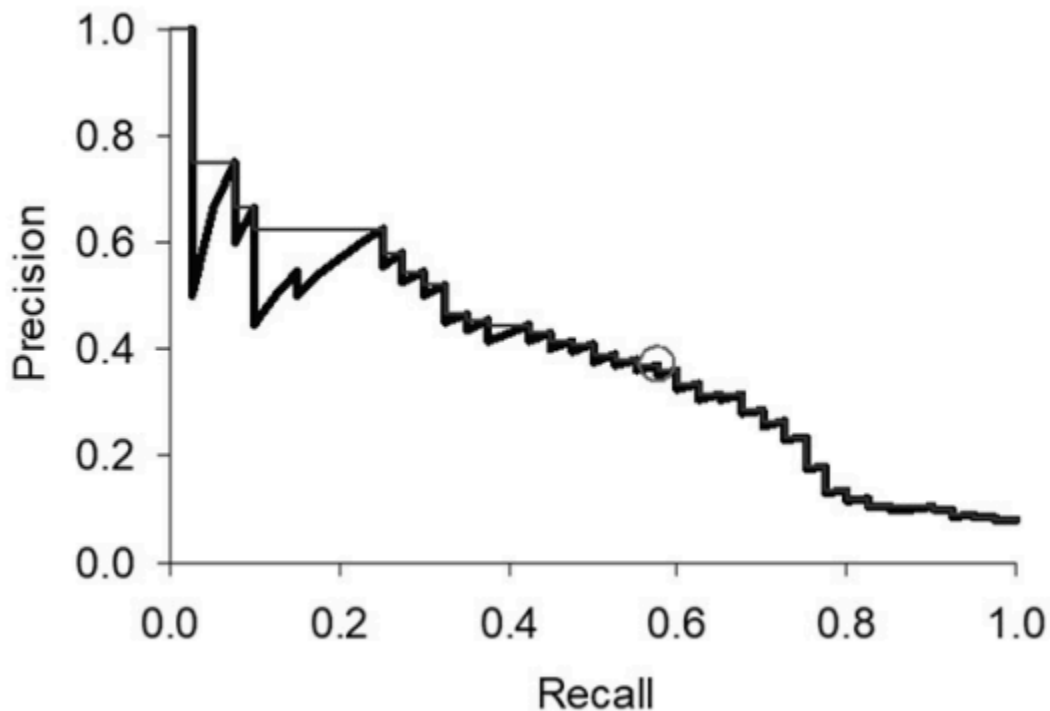
- At $n = 1$: We found 1 relevant doc. $R = 1/6 \approx 0.167$.
- At $n = 4$: We found 3 relevant docs. $R = 3/6 = 0.50$.

2. **Precision (P):** $\frac{\text{Relevant docs found so far}}{\text{Total docs examined (rank } n)}$

- At $n = 1$: 1 relevant out of 1 seen. $P = 1/1 = 1.0$.
- At $n = 4$: 3 relevant out of 4 seen. $P = 3/4 = 0.75$.

Note on Image 1: The note "Never reach 100% recall" means the system stopped returning results (at $n = 14$) before it found the 6th relevant document. This is common in real-world systems.

Precision - recall curve:



When you plot these (R, P) pairs on a graph, you get the jagged line seen in Image 2. This is often called a **Sawtooth Curve**. Here is why it looks like that:

- **The Spikes (Up and Right):** Every time the system hits a **relevant** document, Recall increases (moves right) and Precision usually increases or stays high (moves up).
- **The Drops (Straight Down):** Every time the system hits a **non-relevant** document, Recall stays exactly the same, but Precision drops because the denominator (n) increased. This creates the vertical "teeth" in the graph.

2) Average Precision

Average Precision (AP) is a measure used to evaluate **ranked retrieval results**

It considers:

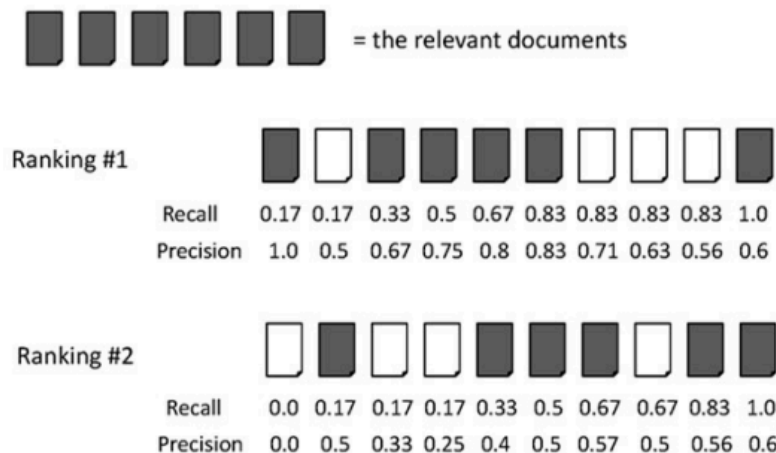
- **Order of documents**
- **Positions of relevant documents**

> It computes the **average of precision values at ranks where relevant documents occur**

> So, it **rewards** systems that place **relevant documents higher in the ranking**

$$AP = \frac{1}{\text{Total Relevant Documents}} \sum (\text{Precision at each relevant document})$$

- We calculate precision **only at ranks where documents are relevant**
- Then take the **average of those precision values**



Ranking #1: $(1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$

Ranking #2: $(0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$

$$\text{Precision (P): } \frac{\text{Relevant docs found so far}}{\text{Total docs examined (rank } n)}$$

2. **Calculate Precision at each Hit:** We only care about the precision at the moment a relevant document is found.

- **Rank 1:** 1 relevant out of 1 seen $\rightarrow P = 1.0$
- **Rank 3:** 2 relevant out of 3 seen $\rightarrow P = 2/3 \approx 0.67$
- **Rank 4:** 3 relevant out of 4 seen $\rightarrow P = 3/4 = 0.75$
- **Rank 5:** 4 relevant out of 5 seen $\rightarrow P = 4/5 = 0.8$
- **Rank 6:** 5 relevant out of 6 seen $\rightarrow P = 5/6 \approx 0.83$
- **Rank 10:** 6 relevant out of 10 seen $\rightarrow P = 6/10 = 0.6$

3. **Final AP Calculation:** Add those 6 precision values together and divide by the total number of relevant documents ($N = 6$).

$$AP_{\text{Ranking 1}} = \frac{1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6}{6} = 0.78$$

👉 Interpretation

- Higher AP means:
 - Relevant documents appear **earlier in the ranking**
- Lower AP means:
 - Relevant documents appear **later or are missing**
- AP captures both:
 - **Precision**
 - **Ranking quality**

AP is Better than Simple Precision ; Normal precision ignores **order of documents**

3) Mean Average Precision MAP

- evaluation metric for **multiple queries**
- It computes the **mean (average) of Average Precision (AP)** over all queries
- While AP evaluates performance for **one query**, MAP evaluates **overall system performance**

$$MAP = \frac{1}{Q} \sum AP(q)$$

- Where:
 - Q = total number of queries
 - $AP(q)$ = Average Precision for each query

Calculation:

For each query:

- Compute **Average Precision (AP)**

Then:

- Take the **average of all AP values**

This gives a **single performance score** for the system

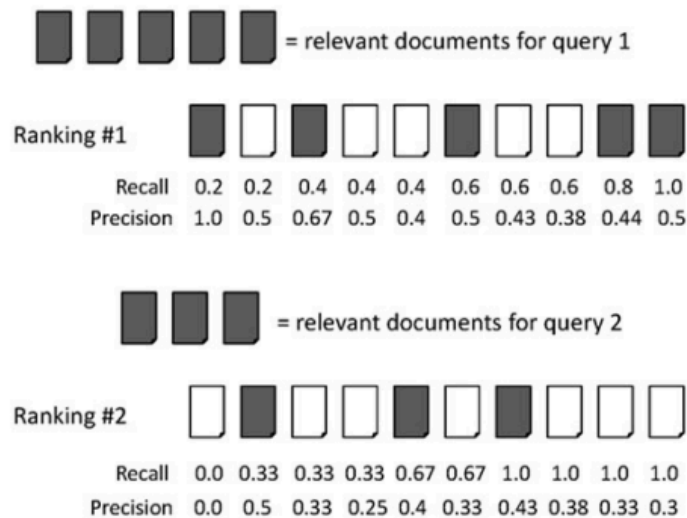
Interpretation:

Higher MAP:

- Relevant documents appear **earlier across queries**

Lower MAP:

- Poor ranking or missing relevant documents



$$\text{average precision query 1} = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$

$$\text{average precision query 2} = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$\text{mean average precision} = (0.62 + 0.44)/2 = 0.53$$

👉 Unretrieved Relevant Documents (Important Assumption)

- If a relevant document is **not retrieved at all**, its contribution to AP is **considered 0**
- This means:
 - Missing relevant documents **directly lowers AP and MAP**
- Example:
 - Total relevant docs = 4
 - System retrieves only 2 relevant docs
 - The remaining 2 contribute **0**, reducing the final score
- This ensures **systems are penalized for missing important results**

👉 MAP is Macro-Averaging

- MAP treats **each query equally**, regardless of:

- Number of relevant documents
- Query difficulty
- Formula reminder:

$$MAP = \frac{1}{Q} \sum AP(q)$$

- Every query contributes **one AP value**, and all are averaged equally
 - This is called **macro-averaging**
-

- MAP is widely used in **IR research papers**
 - Reason:
 - It combines:
 - Precision
 - Ranking quality
 - Performance across multiple queries
 - It gives a **single, reliable performance score**
-

👉 Is MAP Good for Web Search?

- MAP is **not always ideal for web search**
 - Reason:
 - Web users usually:
 - Look at only **top few results**
 - **Do not try to find all relevant documents**
 - MAP assumes users care about **all relevant documents**, which is not realistic for web behavior
 - Metrics like **Precision@k** are often more suitable for web search
-

👉 Assumption of MAP

- MAP assumes that:
 - Users are **interested in finding many or all relevant documents for each query**
 - This is suitable for:
 - Research tasks
 - Legal search
 - Patent search
 - But less suitable for:
 - Casual browsing or quick searches
-

👉 Need for Many Relevance Judgments

- MAP **requires knowing**:
 - **Which documents are relevant for each query**
- This means:
 - A large number of **manual relevance judgments** are needed
- This is:
 - **Time-consuming**
 - **Expensive**
- Especially difficult for **large collections (e.g., web-scale)**

4) Kappa Statistics

- Measures **how much two judges agree**, while **correcting** for **agreement that could happen by chance**
- Simply counting agreement is not enough because some agreement may occur **randomly**
- Kappa gives a **more reliable measure of consistency**

Table 8.2 Calculating the kappa statistic.

		judge 2 relevance		
		yes	no	total
judge 1 relevance	yes	300	20	320
	no	10	70	80
	total	310	90	400

Observed proportion of the times the judges agreed

$$P(A) = (300 + 70)/400 = 370/400 = 0.925$$

Pooled marginals

$$P(\text{nonrelevant}) = (80 + 90)/(400 + 400) = 170/800 = 0.2125$$

$$P(\text{relevant}) = (320 + 310)/(400 + 400) = 630/800 = 0.7878$$

Probability that the two judges agreed by chance

$$P(E) = P(\text{nonrelevant})^2 + P(\text{relevant})^2 = 0.2125^2 + 0.7878^2 = 0.665$$

Kappa statistic

$$\kappa = (P(A) - P(E))/(1 - P(E)) = (0.925 - 0.665)/(1 - 0.665) = 0.776$$

The two judges are **classifying documents for a single query (or information need)**

Each document is labeled as:

- **Relevant (Yes)**
- **Non-relevant (No)**

👉 Given Table (Understanding the Data)

- Two judges classify documents as:
 - Relevant (Yes)
 - Non-relevant (No)

	Judge 2 Yes	Judge 2 No	Total
Judge 1 Yes	300	20	320
Judge 1 No	10	70	80
Total	310	90	400

- Total documents = 400

👉 300 (Yes–Yes)

- Both Judge 1 and Judge 2 said the document is **relevant**
- This is a **perfect agreement**
- These are **correct matches in positive class**

👉 20 (Yes–No)

- Judge 1 said **relevant**
- Judge 2 said **non-relevant**
- This is a **disagreement**
- Shows inconsistency in judging relevance

👉 10 (No–Yes)

- Judge 1 said **non-relevant**
- Judge 2 said **relevant**
- Another **disagreement**
- Similar to above but roles reversed

👉 70 (No–No)

- Both judges said **non-relevant**
- This is also **agreement**
- These are correct matches in negative class

👉 Step 1: Observed Agreement $P(A)$

- This is the proportion of times both judges **actually agreed**

Agreement cases:

- Both said Yes → 300
- Both said No → 70

$$P(A) = \frac{300 + 70}{400} = \frac{370}{400} = 0.925$$

👉 Judges agree **92.5% of the time**

👉 Step 2: Probability of Agreement by Chance $P(E)$

👉 Compute Marginal Probabilities

- Probability of **non-relevant**:

$$P(\text{non-relevant}) = \frac{80 + 90}{800} = \frac{170}{800} = 0.2125$$

- Probability of **relevant**:

$$P(\text{relevant}) = \frac{320 + 310}{800} = \frac{630}{800} = 0.7878$$

- **P(non-relevant)** → no of docs judge 1 voted irrelevant + no of docs judge 2 voted irrelevant
- **P(relevant)** → no of docs judge 1 voted relevant + no of docs judge 2 voted relevant

👉 Chance Agreement Formula

$$P(E) = P(\text{non-relevant})^2 + P(\text{relevant})^2$$

$$P(E) = (0.2125)^2 + (0.7878)^2 = 0.665$$

👉 So, 66.5% agreement could happen by chance

👉 Step 3: Kappa Formula

$$\kappa = \frac{P(A) - P(E)}{1 - P(E)}$$

$$\kappa = \frac{0.925 - 0.665}{1 - 0.665} = \frac{0.26}{0.335} = 0.776$$

👉 Interpretation of Kappa

- $\kappa = 0.776$

👉 This indicates:

- **Strong agreement** between judges
- Agreement is **much better than chance**

👉 Kappa Value Meaning (Important)

- **$\kappa = 1$** → Perfect agreement
- **$\kappa = 0$** → Agreement is just by chance
- **$\kappa < 0$** → Worse than random

👉 0.776 → **High reliability of judgments**

👉 Why Kappa is Important in IR

- Relevance judgments are **subjective**
- Different judges may disagree
- Kappa helps:
 - Measure **consistency**
 - Validate **quality of ground truth data**

📌 Cumulative Gain

Cumulative Gain simply measures:

👉 **How relevant the retrieved documents are overall**

It does this by **adding up the relevance scores** of all documents in the result list.

2. Formula

$$CG = \sum_{i=1}^n rel_i$$

Where:

- **rel_i** = relevance score of document at position *i*
 - **n** = total number of retrieved documents
-

3. Given Example

Query: q

Documents:

$D_1, D_2, D_3, D_4, D_5, D_6$

Relevance scores:

3, 2, 3, 0, 1, 2

Step-by-Step Calculation

$$CG = 3 + 2 + 3 + 0 + 1 + 2 = 11$$

 **Final Answer: CG = 11**

4. Important Property (Very Important!)

 **Changing** the **order** of **documents** does **NOT** affect **CG**

Why?

Because CG only cares about:

- The **total relevance**
 - **NOT** the **position** of documents
-

5. Limitation of CG

This is the biggest drawback:

✗ CG does **not consider ranking (position)**

💡 Why is this a problem?

In real search engines:

- Users care more about **top results**
- A relevant document at position 1 is **more valuable** than at position 6

But CG treats them equally 😞

🔄 6. What came after CG?

Because of this limitation, improved metrics were introduced:

- **Discounted Cumulative Gain (DCG)** → considers position
 - **Normalized DCG (nDCG)** → compares with ideal ranking
-

♦ Discounted Cumulative Gain (DCG)

DCG improves on CG by considering **both relevance AND position (rank)** of documents

👍 DCG measures **how good the ranking is**, giving **higher importance to top-ranked documents**.

- Relevant documents at the **top** → **more valuable** ✓
 - Relevant documents at the **bottom** → less valuable !
-



2. Main Formula

Here is the standard DCG formula:

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2 i}$$



Explanation:

- rel_i = relevance of document at position i
 - $\log_2 i$ = discount factor (reduces importance as position increases)
 - p = rank position up to which we calculate DCG
-



3. Why “Discounted”?

Because of the division:

$$\frac{rel_i}{\log_2 i}$$

👉 As i increases, $\log_2 i$ increases → **value decreases**

👉 So lower-ranked documents contribute less



4. Example

Relevance scores:

(3, 2, 3, 0, 1, 2)

Let's compute DCG up to position 6:

$$DCG = 3 + \frac{2}{\log_2 2} + \frac{3}{\log_2 3} + \frac{0}{\log_2 4} + \frac{1}{\log_2 5} + \frac{2}{\log_2 6}$$

Approx values:

- $\log_2 2 = 1$
- $\log_2 3 \approx 1.58$
- $\log_2 5 \approx 2.32$
- $\log_2 6 \approx 2.58$

$$DCG \approx 3 + 2 + 1.89 + 0 + 0.43 + 0.77 = 8.09$$

👉 **DCG $\approx$ 8.09**

5. Alternative Formula

$$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log(1+i)}$$

6. Why This Alternative?

- $2^{rel_i} - 1$ amplifies highly relevant documents
- Example:
 - $rel = 3 \rightarrow 2^3 - 1 = 7$
 - $rel = 1 \rightarrow 2^1 - 1 = 1$

👉 Big difference → **high relevance gets much more importance**

7. Key Insight (VERY IMPORTANT)

🔥 DCG **rewards**:

- **Highly relevant** documents
 - Appearing **early in the ranking**
-

8. CG vs DCG

Feature	CG	DCG
Considers relevance	✓	✓
Considers position	✗	✓
Realistic ranking measure	✗	✓

! 9. Limitation of DCG

- DCG values are **not normalized**
- **Hard to compare** between different queries

👉 That's why we use **nDCG (Normalized DCG)**

› **solved example: [slide 17]**

This example uses the $\log(i+1)$ version:

$$DCG_p = \sum_{i=1}^p \frac{rel_i}{\log_2(i+1)}$$

i	rel_i	$\log_2(i+1)$	$\frac{rel_i}{\log_2(i+1)}$
1	3	1	3
2	2	1.585	1.262
3	3	2	1.5
4	0	2.322	0
5	1	2.585	0.387
6	2	2.807	0.712

$$DCG_6 = 3 + 1.262 + 1.5 + 0 + 0.387 + 0.712$$

$$DCG_6 = 6.861$$

› Key Observations

✓ Top-ranked documents matter more

- First result contributes **3 (very high)**
- Later ones contribute less due to log discount

! Zero relevance gives zero contribution

- At position 4 → no impact

Contribution decreases with position

- Even $rel = 2$ at position 6 → only **0.712**

◆ Normalized Discounted Cumulative Gain (NDCG)

› allows us to **fairly compare rankings across different queries.**

👉 NDCG measures **how good your ranking is compared to the best possible (ideal) ranking**

- DCG → gives a score
- **NDCG** → tells **how close you are to perfect ranking**



2. Formula

$$NDCG_n = \frac{DCG_n}{IDCG_n}$$

Where:

- DCG_n = DCG of your current ranking
- $IDCG_n$ = DCG of the *ideal ranking* (best possible order)



3. What is Ideal Ranking (IDCG)?



Sort documents in **descending order of relevance**

Example

Original relevance: (3, 2, 3, 0, 1, 2)

Ideal ranking: (3, 3, 2, 2, 1, 0)



4. Step-by-Step Example



Step 1: DCG (already computed)

From previous:

$$DCG = 6.861$$



Step 2: Compute IDCG

Use same formula but with ideal order:

i	rel _i	log ₂ (i+1)	value
1	3	1	3
2	3	1.585	1.892
3	2	2	1
4	2	2.322	0.861
5	1	2.585	0.387
6	0	—	0

$$IDCG \approx 3 + 1.892 + 1 + 0.861 + 0.387 = 7.14$$

✅ Step 3: Compute NDCG

$$NDCG = \frac{6.861}{7.14} \approx 0.96$$

🎯 5. Interpretation

NDCG Value	Meaning
1	Perfect ranking ✅
0.9+	Very good 🔥
0.5	Moderate
< 0.3	Poor ❌

👉 Your result: 0.96 → Excellent ranking

6. Why Normalization is Important

Without normalization:

- **DCG values vary across queries**
- **Hard** to **compare performance**

With NDCG:

- Values are always between **0 and 1**
 - Makes comparison fair 
-

7. Intuition

Think of NDCG like:

 "How close are you to the best possible ranking?"

- If your results = **ideal** → **score = 1**
 - If **badly ordered** → score **closer to 0**
-

8. Summary (CG vs DCG vs NDCG)

Metric	Considers Order	Comparable Across Queries
CG		
DCG		
NDCG		

★ Normalized DCG (Example)

4 documents: d_1, d_2, d_3, d_4

i	Ground Truth		Ranking Function ₁		Ranking Function ₂	
	Document Order	r_i	Document Order	r_i	Document Order	r_i
1	d4	2	d3	2	d3	2
2	d3	2	d4	2	d2	1
3	d2	1	d2	1	d4	2
4	d1	0	d1	0	d1	0
NDCG _{GT} =1.00			NDCG _{RF1} =1.00		NDCG _{RF2} =0.9203	

$$DCG_{GT} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

$$DCG_{RF1} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

$$DCG_{RF2} = 2 + \left(\frac{1}{\log_2 2} + \frac{2}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.2619$$

$$MaxDCG = DCG_{GT} = 4.6309$$

◆ 1. System Quality (Beyond Retrieval Accuracy)

So far, metrics like **precision**, **recall**, **DCG** measure *retrieval quality*.

But in real systems (like search engines), that's **not enough**.

👉 **System Quality** = **How well the system performs overall**



Key Factors of System Quality



1. Indexing Speed

How fast the system **processes** and **stores documents**

- Measured as: **documents per hour**
 - Important because:
 - Web content is constantly growing
 - Systems must update quickly
-



2. Search Speed (Latency)

How fast results are **returned after** a **query**

- Measured in **milliseconds (ms)**
 - Depends on:
 - **Index size**
 - **Algorithm efficiency**
-



3. Query Language Expressiveness

How powerful the **search queries can be**

- **Simple: keyword search**
- **Advanced: Boolean queries, filters, phrases**



Example: "machine learning" AND NOT beginner

More expressive = better control for users

4. Performance on Complex Queries

Can the system handle difficult queries efficiently?

- Multi-keyword queries
 - Filters + ranking
-

5. Collection Size & Coverage

How many documents are **available**

- **Large collection** → **better chances** of **finding relevant info**
 - Also important: **diversity of topics**
-

◆ 2. User Utility (User Satisfaction)

👉 The ultimate goal = **User Happiness** 😊

What affects user utility?

- ✓ **Relevance** (correct results)
 - ✓ **Speed** (fast results)
 - ✓ **Interface** (easy to use)
-

How do we measure it?

We can't measure happiness directly, so we use:

 **Return Rate:** Users coming back again

👉 If **users repeatedly use** the **system** → they are **satisfied**

◆ 3. Evaluation of System Changes (A/B Testing)

When **improving** a **system**, we must **test changes** safely.



What is A/B Testing?

Compare two versions of a system

- A = **current** system (**control**)
- B = **new** system (**variant**)



How it works

- **90–99%** users → **old** system
- **1–10%** users → **new** system

👉 Then **compare performance**



What do we measure?

✓ Click-through rate (CTR)

- Do users click results?

✓ Clickstream data

- User behavior patterns

👉 If users interact more → new system is better ✓



Important Rule

🔥 Change **ONLY ONE** thing at a time

Why?

To **know exactly what caused improvement**

◆ 4. Search Snippets

👉 **Short previews** shown **under** each **search result**



Why are snippets important?

- Help users decide:
 - Which result to click
 - Whether it's relevant

Types of Snippets

♦ 1. Static Snippets

- Same summary for every query
- Pre-generated

👉 Example:

- First few lines of a document

♦ 2. Dynamic Snippets (Query-dependent)

- Generated based on the query
- Highlight relevant parts

👉 Example:

Search: *"AI applications"*

Snippet shows part containing those words

🔑 KWIC (Keyword In Context)

Shows **keyword** with **surrounding text**

Example:

... modern AI applications are transforming healthcare ...



Final Summary

- **System Quality** → speed, scalability, capability
- **User Utility** → user satisfaction & return rate
- **A/B Testing** → test improvements safely
- **Snippets** → help users choose results



Food for thoughts

1. What is the common scheme for evaluation of information retrieval experiments? explain.

Answer:

An IR system is evaluated using a **test collection**, which includes three key parts:

4. **Document Collection** – a set of documents
5. **Queries (Information Needs)** – what users are searching for
6. **Relevance Judgments** – which documents are relevant

These three together allow us to **measure how well a system retrieves useful information**.

Evaluation metrics like **precision, recall, F1, MAP, DCG** are used to measure performance.

👉 This approach ensures **objective, repeatable, and comparable evaluation** of different IR systems.

2. What is the relationship between the value of F1 and the break-even point?

Answer:

- **F1 score** is the **harmonic mean** of **precision** and **recall**.
- The **break-even point** is where **precision = recall**.

👉 When precision equals recall:

- **F1 = Precision = Recall**

So, the **F1 score at the break-even point equals that common value**, making them closely related.

3. Consider an information need for which there are 4 relevant documents in the collection. Contrast two systems run on this collection. Their top 10 results are judged for relevance as follows (the leftmost item is the top ranked search result):

System 1: R N R N N N N R R

System 2: N R N N R R R N N N

(a) What is the MAP of each system? Which has a higher MAP?

Answer:

Relevant documents = 4

System 1: R N R N N N N R R

Precision at relevant positions:

- Rank 1 $\rightarrow 1/1 = 1$
- Rank 3 $\rightarrow 2/3 \approx 0.67$
- Rank 9 $\rightarrow 3/9 \approx 0.33$
- Rank 10 $\rightarrow 4/10 = 0.4$

$$\text{MAP}_1 = (1 + 0.67 + 0.33 + 0.4) / 4 \approx \mathbf{0.60}$$

System 2: N R N N R R R N N N

Precision at relevant positions:

- Rank 2 $\rightarrow 1/2 = 0.5$
- Rank 5 $\rightarrow 2/5 = 0.4$
- Rank 6 $\rightarrow 3/6 = 0.5$
- Rank 7 $\rightarrow 4/7 \approx 0.57$

$$\text{MAP}_2 \approx (0.5 + 0.4 + 0.5 + 0.57) / 4 \approx \mathbf{0.49}$$

👉 **System 1 has higher MAP**

(b) Does this result intuitively make sense? What does it say?

Answer:

Yes, it makes sense.

👉 System 1 retrieves a relevant document at **rank 1**, which is very important.

👉 **MAP rewards early retrieval** of relevant documents.

✓ Conclusion:

- **Higher-ranked relevant** documents **improve MAP** significantly
-

(c) What is the R-precision of each system?

Answer:

$R = 4 \rightarrow$ **look at top 4 results**

- System 1: R N R N $\rightarrow 2$ relevant $\rightarrow$ **R-precision = $2/4 = 0.5$**
- System 2: N R N N $\rightarrow 1$ relevant $\rightarrow$ **R-precision = $1/4 = 0.25$**

👉 **System 1 is better in both MAP and R-precision**

4. What is the advantage of using the harmonic mean (F1) rather than arithmetic mean?

Answer:

- The **harmonic mean penalizes extreme values** more than arithmetic mean

👉 Example:

- Precision = 1, Recall = 0
- Arithmetic mean = 0.5 ❌ (misleading)
- F1 = 0 ✅ (correctly reflects poor performance)

✓ Advantage:

- Ensures both precision and recall must be **high and balanced**
-

5. What is Fall-out for a system? when is it useful?

Answer:

- **Fall-out** measures the proportion of **non-relevant documents retrieved**

$$Fallout = \frac{Non-relevant\ retrieved}{Total\ non-relevant}$$

👉 Useful when:

- There are **many non-relevant documents** (large collections)
 - We want to **minimize false positives**
-

6. Why evaluation of ranked retrieval is more challenging? Explain.

Answer:

Because:

- **Order of results matters**
- Users care more about **top results**
- **Multiple relevance levels** (not just relevant/non-relevant)
- Difficult to define a single perfect metric

👉 Requires advanced metrics like:

- **MAP**
 - **DCG / NDCG**
-

7. What are some of the drawbacks of cumulative gain?

Answer:

- Does **not consider ranking order** ❌
 - Treats all positions equally
 - Cannot distinguish between good and bad rankings
 - Not realistic for user behavior
-

8. How NDCG overcomes drawbacks of ranked retrieval? Explain.

Answer:

NDCG improves evaluation by:

- ✓ Considering **position (ranking)** using discounting
- ✓ Giving **more weight** to **top results**
- ✓ Handling **graded relevance** (not just binary)
- ✓ **Normalizing scores** (0 to 1) for fair comparison

👉 Result:

- More realistic and widely used in search engines
-

9. What is so good about A/B Testing for IR systems?

Answer:

A/B Testing:

- Compares **current vs new system**
- Uses **real user behavior** (clicks, interaction)
- **Changes one variable at a time**

✓ Advantages:

- Real-world evaluation
- Detects small improvements
- Reduces risk before full deployment

👉 Helps continuously improve IR systems based on actual usage

10. Differentiate between dynamic and static summaries.

Answer:

Feature	Static Summary	Dynamic Summary
Definition	Same for all queries	Depends on query
Generation	Precomputed	Generated at runtime
Relevance	General	Query-specific
Example	First lines of document	Keyword-based snippet

👉 Dynamic summaries are better because:

- They show **why a document is relevant**

› book ex:

? **Exercise 8.10 [★★]** Below is a table showing how two human judges rated the relevance of a set of twelve documents to a particular information need (0 = nonrelevant, 1 = relevant). Let us assume that you've written an IR system that for this query returns the set of documents {4, 5, 6, 7, 8}.

docID	Judge 1	Judge 2
1	0	0
2	0	0
3	1	1
4	1	1
5	1	0
6	1	0
7	1	0
8	1	0
9	0	1
10	0	1
11	0	1
12	0	1

- Calculate the kappa measure between the two judges.
- Calculate precision, recall, and F_1 of your system if a document is considered relevant only if the two judges agree.
- Calculate precision, recall, and F_1 of your system if a document is considered relevant if either judge thinks it is relevant.

a. Calculate the kappa measure between the two judges.

Step 1: Observed Agreement (P_o)

- Both say 0 → docs 1,2 → 2
- Both say 1 → docs 3,4 → 2

Total agreement = 4

$$P_o = \frac{4}{12} = 0.333$$

Step 2: Expected Agreement (P_e)

- Judge 1: 6 ones, 6 zeros $\rightarrow$ 0.5 each
- Judge 2: 6 ones, 6 zeros $\rightarrow$ 0.5 each

$$P_e = (0.5 \times 0.5) + (0.5 \times 0.5) = 0.25 + 0.25 = 0.5$$

Step 3: Kappa

$$\kappa = \frac{P_o - P_e}{1 - P_e} = \frac{0.333 - 0.5}{1 - 0.5} = \frac{-0.167}{0.5} = -0.334$$

👉 Final Answer: $\kappa \approx -0.334$

(Indicates poor agreement, worse than chance)

[do this khud se table banakr]

a. Kappa Measure (Using Contingency Table)

◆ Step 1: Build Agreement Table

	Judge 2 = 1	Judge 2 = 0	Row Total
Judge 1 = 1	2	4	6
Judge 1 = 0	4	2	6
Column Total	6	6	12

◆ Step 2: Observed Agreement (P_o)

Agreement happens when both judges agree:

- Both say 1 → 2
- Both say 0 → 2

$$P_o = \frac{2 + 2}{12} = \frac{4}{12} = 0.333$$

◆ Step 3: Expected Agreement (P_e)

$$P_e = \left(\frac{6}{12} \times \frac{6}{12} \right) + \left(\frac{6}{12} \times \frac{6}{12} \right)$$

$$P_e = 0.25 + 0.25 = 0.5$$

◆ Step 4: Kappa Formula

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

$$\kappa = \frac{0.333 - 0.5}{1 - 0.5} = \frac{-0.167}{0.5} = -0.334$$

Pe is supposed to be squared?

b. Precision, Recall, F1 (relevant only if BOTH judges agree)

Answer:

Relevant documents (both say 1):

👉 {3, 4}

System output:

👉 {4, 5, 6, 7, 8}

Step 1: True Positives (TP)

- Only doc 4 is relevant

👉 TP = 1

Step 2: Metrics

$$Precision = \frac{1}{5} = 0.2$$

$$Recall = \frac{1}{2} = 0.5$$

$$F1 = \frac{2 \times 0.2 \times 0.5}{0.2 + 0.5} = \frac{0.2}{0.7} \approx 0.286$$

👉 Final Answers:

- Precision = 0.2
- Recall = 0.5
- F1 $\approx$ 0.286

c. Precision, Recall, F1 (relevant if EITHER judge says relevant)

Answer:

Relevant documents (union):

👉 {3,4,5,6,7,8,9,10,11,12} → 10 docs

System output:

👉 {4,5,6,7,8}

Step 1: True Positives (TP)

All retrieved docs are relevant

👉 TP = 5

Step 2: Metrics

$$Precision = \frac{5}{5} = 1.0$$

$$Recall = \frac{5}{10} = 0.5$$

$$F1 = \frac{2 \times 1 \times 0.5}{1 + 0.5} = \frac{1}{1.5} \approx 0.667$$

👉 Final Answers:

- Precision = 1.0
- Recall = 0.5
- F1 $\approx$ 0.667

1. What is R-Precision?

› Balances precision and recall

👉 **R-Precision = Precision at rank R**

Where:

- **R = total number of relevant documents** for a query in the entire collection
-



2. Formula

$$R\text{-Precision} = \frac{\text{Number of relevant documents in top } R \text{ results}}{R}$$



4. Example

Suppose:

- Total relevant documents (**R**) = 4
- System returns ranking:

R N R N N R R N N N



Step 1: Look only at top R = 4 results

Top 4:

R N R N

✓ Step 2: Count relevant in top 4

- Relevant = 2
-

✓ Step 3: Compute

$$R\text{-Precision} = \frac{2}{4} = 0.5$$
